AUTOMATIZACIÓN DE PRUEBAS DE SOFTWARE MEDIANTE CUCUMBER Y LENGUAJE GHERKIN: INTEGRACIÓN CON C# Y PYTHON PARA LA VALIDACIÓN DE ESCENARIOS

INGENIERÍA DE REQUERIMIENTOS
DOCENTE: Dr. GUERRERO ULLOA GLEISTON CICERON
TA-INFORME PARCIAL
ESTUDIANTE: ARIAS MAYBELIN

El presente informe parcial describe la implementación de pruebas automatizadas utilizando Cucumber y el lenguaje Gherkin, integrados con C# y Python, para validar el Sistema Integral de Gestión, Control y Seguimiento de Lectura Interna en la Biblioteca Universitaria mediante Códigos QR e Integración de Inteligencia Artificial. Se realizaron las etapas de instalación, configuración, definición de escenarios y ejecución de pruebas, con el propósito de verificar el cumplimiento de los requerimientos funcionales del sistema. La automatización permitió mejorar la eficiencia del proceso de validación y garantizar la calidad y confiabilidad del software desarrollado.

INTRODUCCIÓN

El presente informe se enfoca en la aplicación de la herramienta Cucumber y el lenguaje Gherkin para la automatización de pruebas del Sistema Integral de Gestión, Control y Seguimiento de Lectura Interna en la Biblioteca Universitaria mediante Códigos QR e Integración de Inteligencia Artificial. Este sistema tiene como finalidad gestionar y controlar el acceso y seguimiento de la lectura interna dentro de la biblioteca universitaria, utilizando tecnologías modernas que permiten optimizar el control, mejorar la trazabilidad de la información y fortalecer la gestión bibliotecaria.

Para el cumplimiento de este propósito, se realizó la instalación y configuración de Cucumber en entornos de desarrollo compatibles con C# y Python, así como la creación de escenarios de prueba utilizando el lenguaje Gherkin, el cual permite definir el comportamiento esperado del sistema de forma clara, estructurada y comprensible. Posteriormente, se ejecutaron las pruebas automatizadas con el objetivo de validar el correcto funcionamiento de las funcionalidades implementadas.

La automatización de pruebas mediante Cucumber contribuye a mejorar la eficiencia del proceso de validación, reducir errores, optimizar el tiempo de prueba y asegurar la calidad del sistema desarrollado. De esta manera, este informe presenta de forma estructurada las actividades realizadas, los procedimientos aplicados y los resultados obtenidos durante el proceso de automatización de pruebas del sistema, garantizando el cumplimiento de los estándares de calidad establecidos en el desarrollo de software.

1. BASES TEÓRICAS DE LA AUTOMATIZACIÓN DE PRUEBAS EN ENTORNOS C# Y PYTHON UTILIZANDO CUCUMBER

En la actualidad, las pruebas manuales han dejado de ser suficientes frente a la creciente complejidad de los sistemas de software, dando paso a la automatización como un estándar indispensable de calidad. Herramientas como Cucumber y el lenguaje Gherkin permiten validar de forma precisa, rápida y estructurada el comportamiento del sistema mediante su integración con C# y Python. Este enfoque garantiza la confiabilidad y correcto funcionamiento del Sistema Integral de Gestión y Seguimiento de Lectura en la Biblioteca Universitaria.

El propósito del presente informe es documentar el proceso de instalación, configuración y ejecución de pruebas automatizadas en C# y Python bajo el enfoque BDD con Cucumber. El alcance incluye:

1. La instalación de entornos de desarrollo en Visual Studio (.NET/C#) y Visual Studio Code (Python).
2. La escritura de escenarios en lenguaje Gherkin.
3. La implementación de definiciones de pasos (Step Definitions) en ambos lenguajes.
4. La ejecución y validación de pruebas automatizadas.

Este informe busca servir como referencia metodológica para proyectos de software que requieren pruebas ágiles y verificables[1], [2], [4].

La principal problemática en proyectos de desarrollo ágil radica en la dificultad de mantener una comunicación clara entre equipos técnicos y no técnicos. Los requisitos suelen ser ambiguos o incompletos, lo que genera errores costosos en etapas avanzadas[5].

El uso de lenguaje natural estructurado (Gherkin) permite cerrar esta brecha, ya que los escenarios escritos pueden ser revisados por clientes, analistas y desarrolladores antes de su implementación. En este sentido, BDD se convierte en un marco metodológico que reduce la ambigüedad y aumenta la trazabilidad de los requisitos[2].

2. ACOPLAMIENTO DE CUCUMBER (LENGUAJE GHERKIN) CON LA PLATAFORMA DE C#.

El acoplamiento de Cucumber con la plataforma C# permite implementar pruebas automatizadas mediante el uso del lenguaje Gherkin, facilitando la definición clara y estructurada de los escenarios de prueba. Este proceso comprende la instalación y configuración de las herramientas necesarias, la escritura de escenarios que describen el comportamiento esperado del sistema y la ejecución de pruebas para validar su correcto funcionamiento. De esta manera, se garantiza la verificación eficiente de los requerimientos y la calidad del sistema desarrollado.

2.1 Instalación

La instalación corresponde al proceso de configuración de las herramientas necesarias para implementar Cucumber en el entorno de desarrollo C#. Esta etapa permite preparar el sistema para la creación y ejecución de pruebas automatizadas, garantizando su correcto funcionamiento.

2.1.1 Instalación de la extensión Reqnroll

Inicialmente, se abrió Visual Studio y se ingresó al administrador de extensiones para gestionar los complementos disponibles. A continuación, se localizó la extensión Reqnroll y se realizó su instalación utilizando el archivo instalador VSIX. **Figura 1** Una vez finalizada la instalación, se reinició el entorno de desarrollo con el propósito de asegurar la correcta integración y funcionamiento de la extensión.

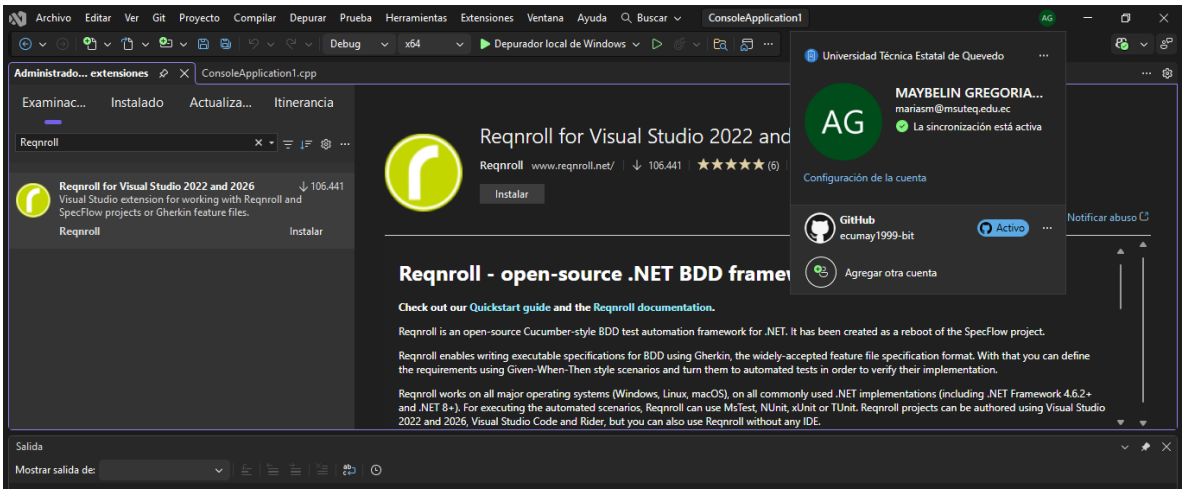


FIGURA 1
Instalación de la extensión Reqnroll

Una vez completada la instalación de la extensión Reqnroll y configurado correctamente el entorno de desarrollo en Visual Studio, el sistema quedó preparado para la creación y ejecución de pruebas automatizadas. **Figura 2** Esta configuración permite utilizar el lenguaje Gherkin para definir escenarios de prueba y validar el funcionamiento del sistema de manera eficiente y estructurada.

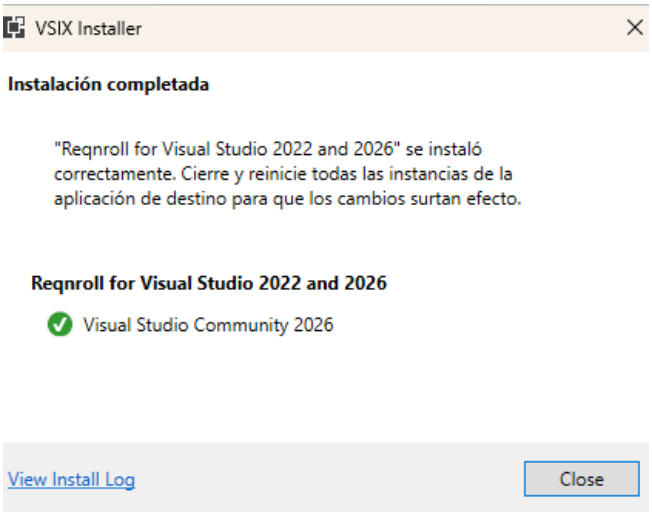


FIGURA 2
Instalación lista

2.2 Escritura de escenarios

La escritura de escenarios consiste en la definición estructurada de los comportamientos esperados del sistema mediante el lenguaje Gherkin. Esta etapa permite describir de forma clara y comprensible las funcionalidades que serán validadas mediante pruebas automatizadas.

Visual Studio es un entorno de desarrollo integrado que permite crear, configurar y ejecutar aplicaciones utilizando diferentes lenguajes de programación, entre ellos C#. **Figura 3** Este entorno proporciona herramientas especializadas para el desarrollo, depuración y automatización de pruebas, facilitando la integración con extensiones como Reqnroll para la validación automatizada del sistema.

FIGURA 3
Presentación del entorno de desarrollo Visual Studio

Se generó un nuevo proyecto utilizando la plantilla proporcionada por Reqnroll. **Figura 4** En la etapa de configuración inicial, se eligió el framework .NET 8.0 junto con NUnit como entorno de pruebas, lo que permitió habilitar la ejecución de pruebas automatizadas a través de la herramienta Test Explorer.

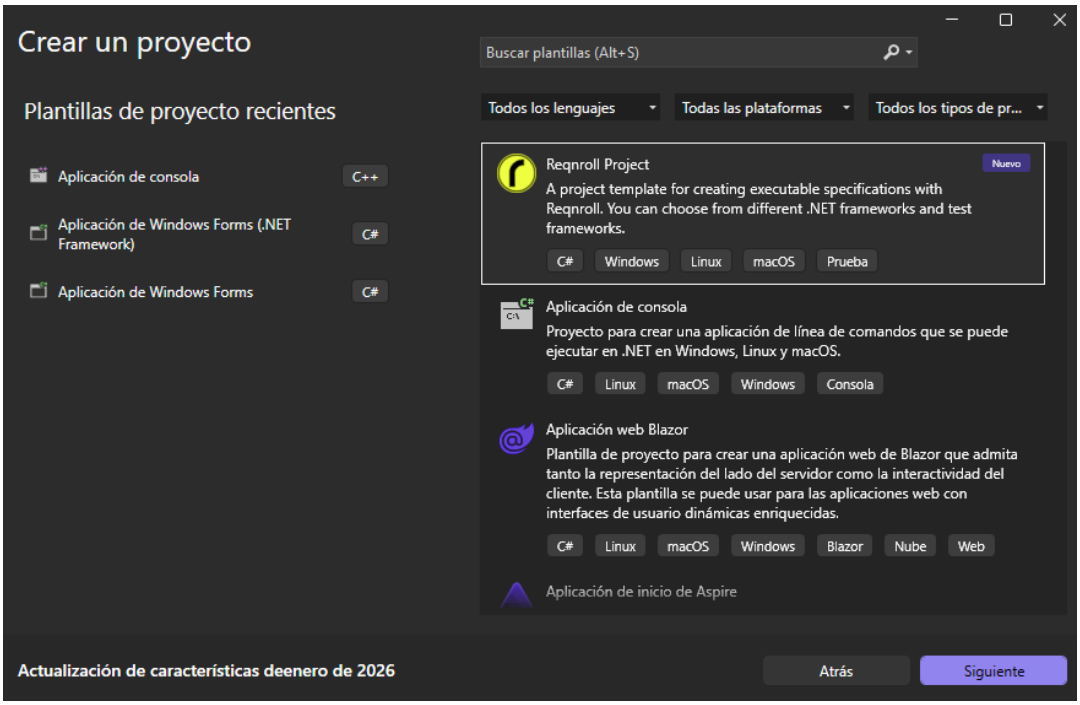


FIGURA 4
Nuevo proyecto Reqnroll

Dentro del proyecto nuevo en la Carpeta *.feature*, se añadió un nuevo elemento llamado archivo de texto con el cual se utiliza para definir los escenarios de prueba mediante el lenguaje Gherkin. **Figura 5 y 6.**

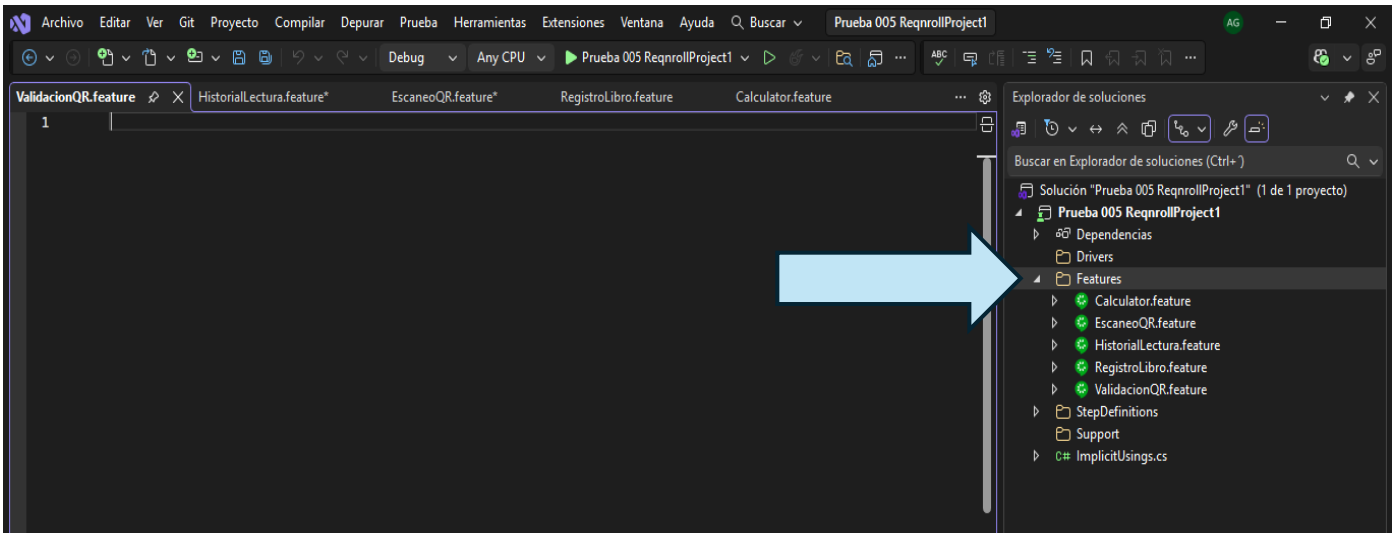


FIGURA 5
Carpeta .feature

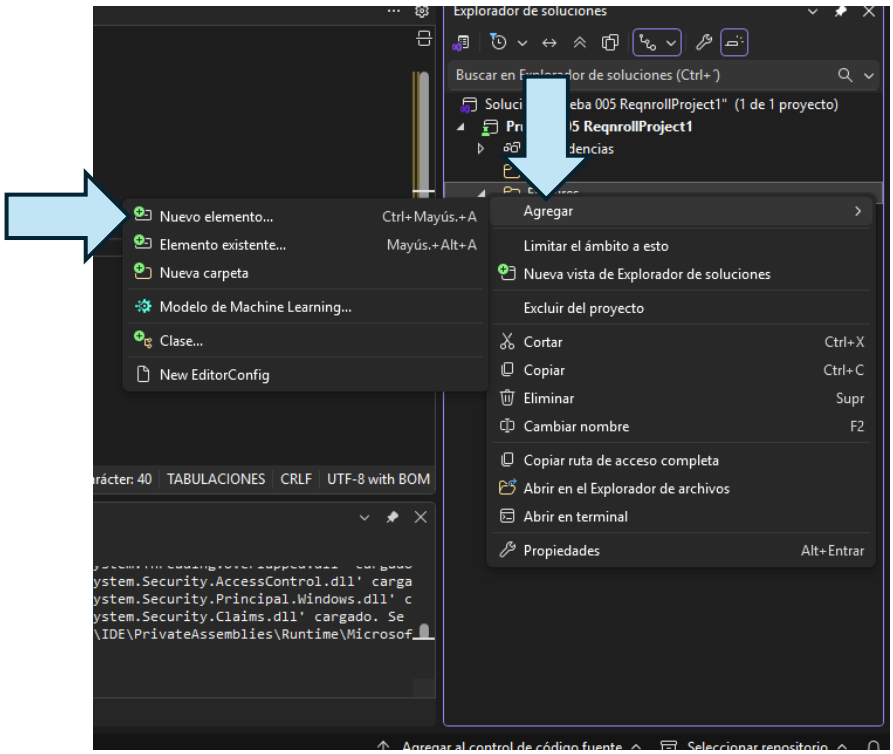
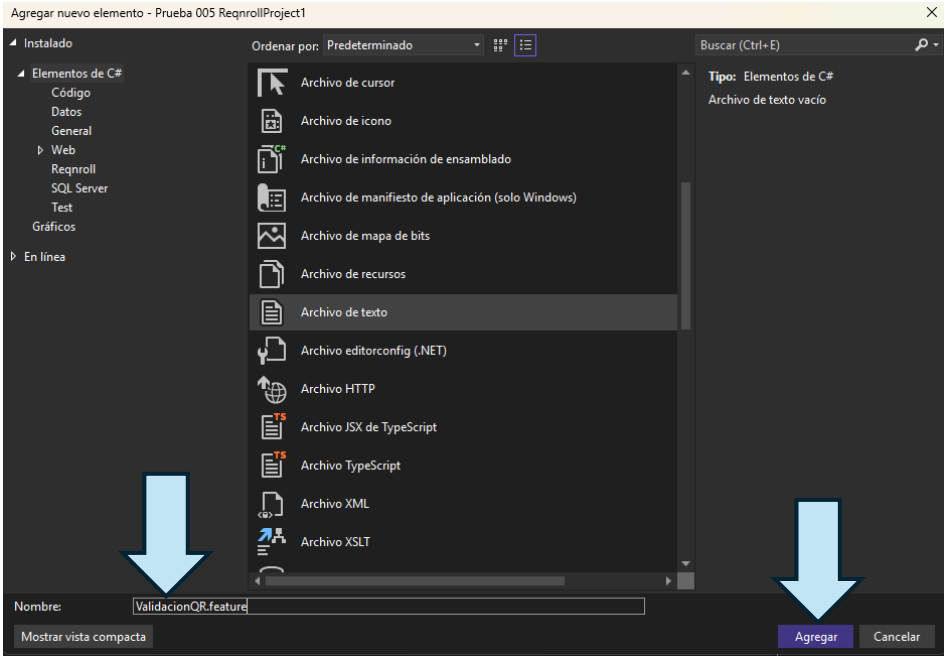


FIGURA 6
Creación de un nuevo elemento en la carpeta Feature.

La definición de escenarios constituye una etapa fundamental en la automatización de pruebas, ya que permite describir de forma clara y estructurada el comportamiento esperado del sistema mediante el lenguaje Gherkin. **Figura 6** Estos escenarios facilitan la validación de las funcionalidades implementadas y aseguran el cumplimiento de los requerimientos establecidos.



Dentro del proyecto, **Figura 7** al utilizar la opción **"Define Steps"**, Reqnroll creó de forma automática una clase en C# que contiene los métodos asociados a cada uno de los pasos especificados en el archivo *.feature*. **Figura 8 , Figura 9** Estos métodos se generan inicialmente con la excepción **PendingStepException()**, lo que indica que es necesario completar su implementación para definir el comportamiento de cada paso.

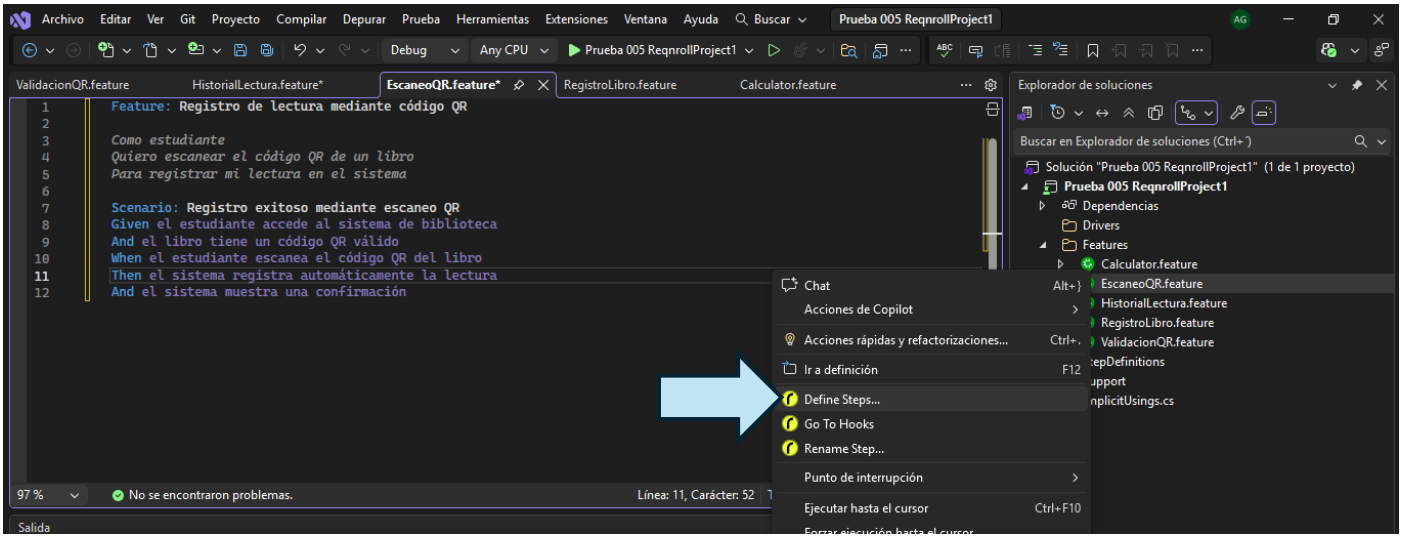


FIGURA 7
Al dar clic en la pantalla del proyecto se despliega la lista que vemos en la imagen y damos clic en Define Steps

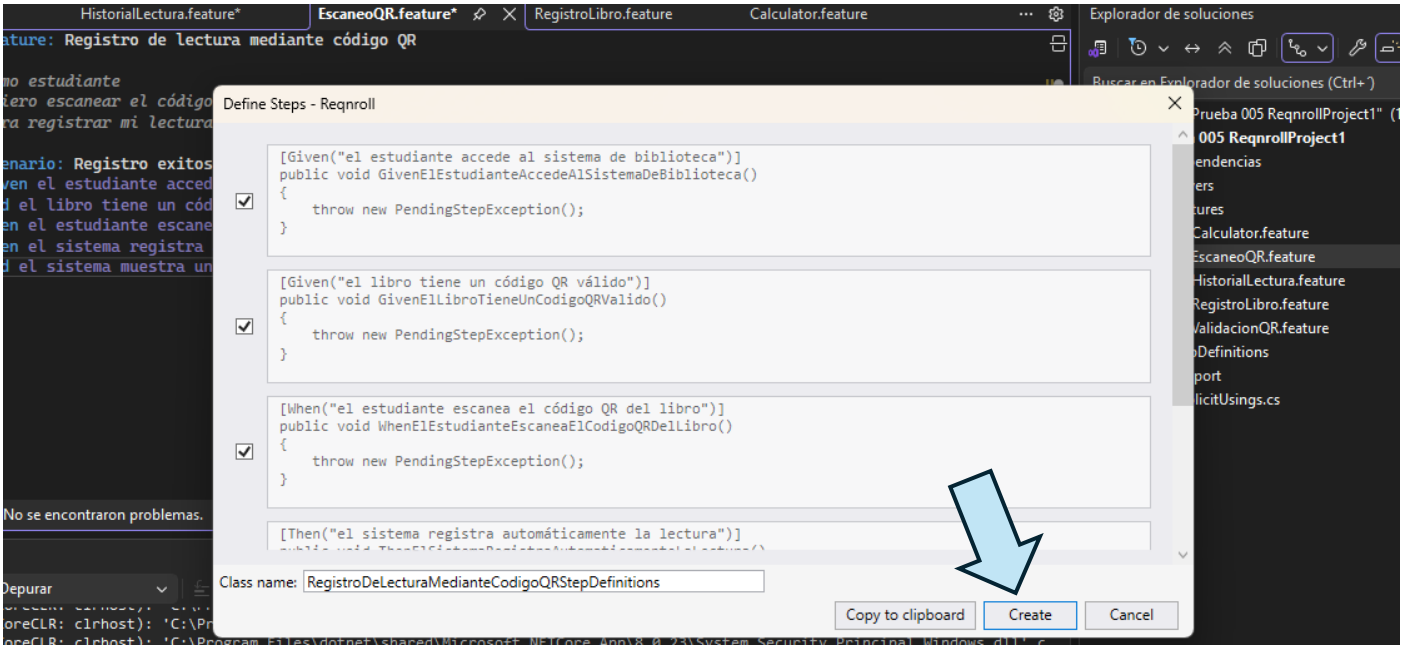


FIGURA 8
Dar clic en Create

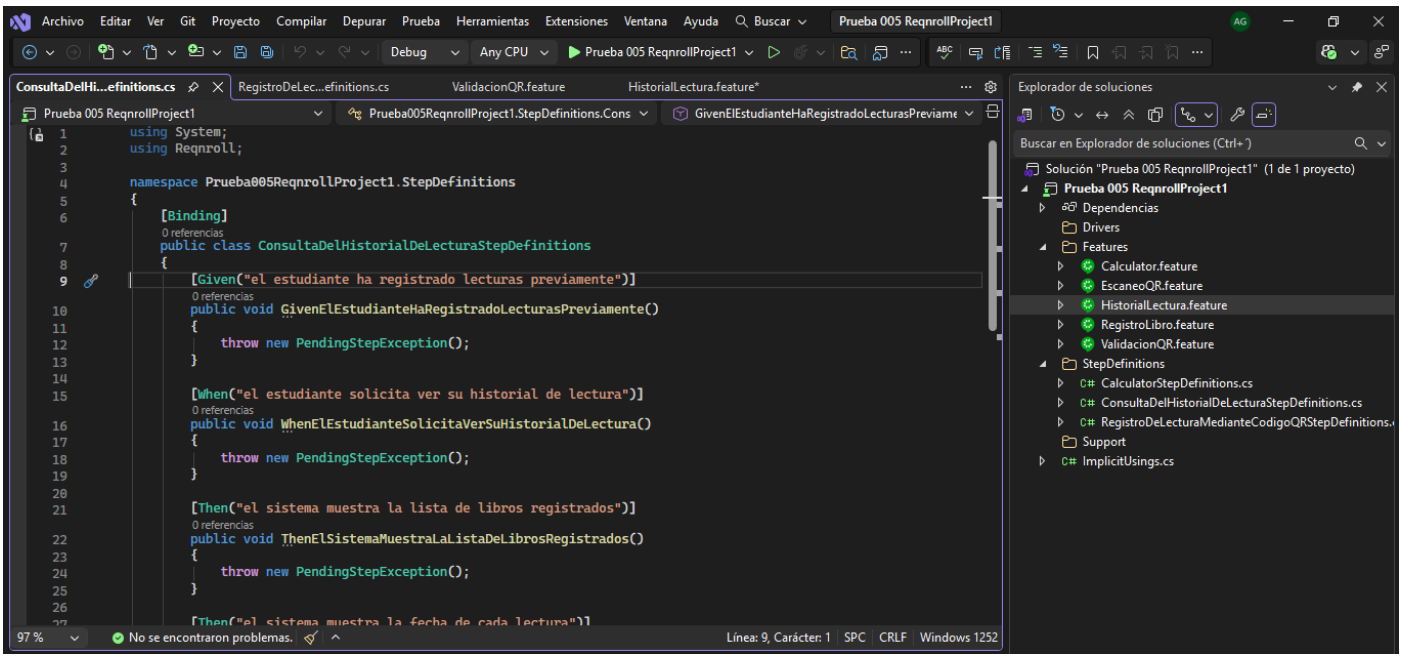
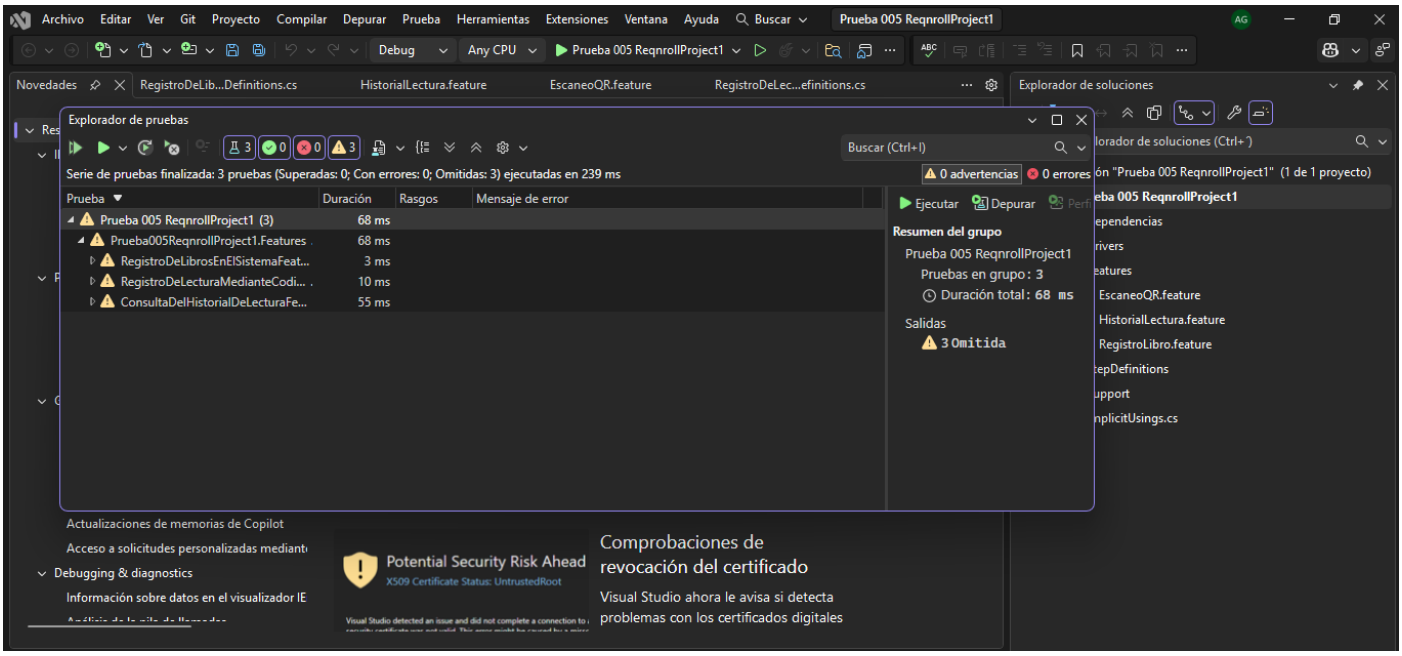


FIGURA 9
StepDefinitions

2.3 Prueba y validación

La prueba y validación **Figura 10** corresponden a la ejecución de los escenarios definidos con el fin de verificar el correcto funcionamiento del sistema. Este proceso permite identificar posibles errores y asegurar el cumplimiento de los requerimientos establecidos.

Después de definir e implementar los pasos correspondientes en el archivo de escenarios, se procedió a ejecutar las pruebas automatizadas utilizando el Test Explorer de Visual Studio. Durante este proceso, el sistema ejecutó cada uno de los pasos definidos en el archivo *.feature*, verificando que el comportamiento implementado coincidiera con el resultado esperado.



Como resultado de la ejecución, el escenario se completó correctamente, lo que confirmó el adecuado funcionamiento de la funcionalidad evaluada. Este proceso permitió validar el cumplimiento de los requerimientos definidos y comprobar la correcta integración de Cucumber (Reqnroll) con la plataforma C#.

RESULTADOS

Los escenarios definidos en Gherkin fueron ejecutados exitosamente en ambos entornos.

1. En C#, SpecFlow garantizó la trazabilidad entre escenarios y código de prueba.

La salida de ejecución mostró todos los pasos validados con estado Passed, confirmando el correcto acoplamiento entre especificación y desarrollo[2], [3].

DISCUSIÓN

Los resultados confirman que:

1. SpecFlow ofrece una integración sólida en el ecosistema .NET, con soporte para frameworks como xUnit y NUnit, ideal para proyectos empresariales que ya utilizan C#.
2. Ambos frameworks permiten que los escenarios en Gherkin sean independientes del lenguaje de programación, lo que fomenta la colaboración interdisciplinaria.

Así mismo, la metodología BDD demostró ventajas sobre pruebas unitarias tradicionales: mayor claridad en requisitos, mejor trazabilidad y reducción de costos por errores tardíos[4], [5].

BIBLIOGRAFIA

[1] S. Santos, T. Pimentel, F. G. Rocha, and M. S. Soares, "Using Behavior-Driven Development (BDD) for Non-Functional Requirements," *Software 2024, Vol. 3, Pages 271-283*, vol. 3, no. 3, pp. 271–283, Jul. 2024, doi: 10.3390/SOFTWARE3030014.

[2] M. Z. Anjum, S. T. Mac Mahon, and F. McCaffery, "Development of Health Software using Behaviour Driven Development - BDD," *International Conference on Model-Driven Engineering and Software Development*, pp. 149–157, 2020, doi: 10.5220/0008984201490157.

[3] "SpecFlow: a Free and Open Source BDD Framework for .NET | by Mahesh Patidar | Globant | Medium." Accessed: Aug. 30, 2025. Available: https://medium.com/globant/specflow-a-free-and-open-source-bdd-framework-for-net-55cc579a3a2b

[4] "(PDF) Designing and Implementing Robust Test Automation Frameworks using Cucumber BDD and Java." Accessed: Aug. 30, 2025. . Available: https://www.researchgate.net/publication/392085234_Designing_and_Implementing_Robust_Test_Automation_Frameworks_using_Cucumber_BDD_and_Java

[5] "Puntos de datos: Diseño controlado por comportamiento con SpecFlow | Microsoft Learn." Accessed: Aug. 30, 2025. Available: https://learn.microsoft.com/es-es/archive/msdn-magazine/2013/july/data-points-behavior-driven-design-with-specflow